ULTIMATE CRYPTO BACKTESTER PRO

Complete Methodology & Technical Reference

Live Binance | Multi-TF Signals | Leverage & Risk Control | Smart Optimizer

Version 2.0 | March 2026

Architecture: Monolithic SPA (~20,500 lines) | Frontend: Vanilla JS + Chart.js + LightweightCharts
Backend: Node.js/Express on Railway | Exchange: BloFin API | Data: Binance, DexScreener, RugCheck

8 Tabs | 400+ Functions | 7 Technical Indicators | AI Probability Engine

TABLE OF CONTENTS

1. Architecture Overview 6
1.1 Technology Stack 6
1.2 Tab Structure 6
1.3 Global Systems 6
1.3.1 Authentication (line 674-927) 7
1.3.2 Live Price Ticker (line 4459-4474) 7
1.3.3 Market Cycle Dashboard (line 3194-3872) 7
2. Backtester Tab 8
2.1 Data Loading 8
2.1.1 fetchKlines() (line 4662) 8
2.1.2 fetchBlofinCandles() (line 4514) 8
2.1.3 fetchStockKlines() (line 4550) 8
2.1.4 resampleCandles() (line 4604) 8
2.2 Technical Indicators 8
2.2.1 Core Math Functions (line 4059-4068) 8
2.2.2 Signal Generators (line 4079-4211) 9
2.3 Signal Combination 9
2.3.1 combineSignals() (line 5119) 9
2.3.2 combineCrossTFSignals() (line 5074) 9
2.3.3 buildTimelineIndex() (line 5055) 10
2.4 Backtest Engine 10
2.4.1 backtest() (line 4211) 10
2.4.2 Position Sizing Modes 10
2.4.3 walkForwardTest() (line 4386) 10
2.5 Optimizers 10
2.5.1 Leverage Optimizer (line 7694) 10
2.5.2 SL/TP Optimizer (line 7759) 11
2.5.3 DCA Comparison (line 8030) 11
2.6 Market Quality & Regime Detection 11
2.6.1 buildSignalQuality() (line 4877) 11
2.6.2 detectRegime() (line 4905) 11
2.7 Charts & Visualization 11
2.7.1 drawPriceChart() (line 6159) 11
2.7.2 drawIndicatorSubPanels() (line 6336) 11

2.7.3 drawPNLChart() (line 7170) 11
2.7.4 drawTradesTable() (line 7202) 11
2.8 Pine Script Export (line 7396) 12
2.9 AI Strategy Analyst (line 5819) 12
2.10 Funding Rate Monitor (line 7004-7149) 12
3. Degen Scanner Tab 13
3.1 Data Sources & Scanning 13
3.1.1 runClientDegenScan() (line 9655) 13
3.1.2 dexFetch() (line 9194) 13
3.1.3 rugFetch() (line 9205) 13
3.1.4 parseRugReport() (line 9220) 13
3.2 Token Scoring - clientScoreToken() (line 9338) 13
3.2.1 Liquidity (line 9369) 13
3.2.2 Volume/Market Cap Ratio (line 9375) 13
3.2.3 Volume Acceleration (line 9387) 14
3.2.4 Wash Trade Detection (line 9399) 14
3.2.5 Buy Pressure (line 9418) 14
3.2.6 Age Sweet Spot (line 9432) 14
3.2.7 Market Cap Band (line 9441) 15
3.2.8 Additional Scoring Factors 15
3.3 Probability Calibration - calcMemeProbability() (line 10327) 15
3.4 RugCheck Integration 15
3.5 Token Tiers & Display 16
3.5.1 buildDegenTiers() (line 9606) 16
3.6 Smart Money Detection - checkSmartMoney() (line 16496) 16
3.7 Auto-Trade System - checkDegenAutoTrade() (line 16200) 16
3.7.1 Safety Gates (in order of evaluation) 16
3.7.2 Position Sizing & Exit Strategy 17
3.8 Adaptive Learning Engine (line 16310-16475) 17
3.9 Sub-tabs 17
4. Market Intel Tab 19
4.1 Market Overview - renderOverview() (line 11073) 19
4.2 Coin Screener - renderScreener() (line 11207) 19
4.3 AI Top Picks - generateAIPicks() (line 11280) 19
4.4 Fear & Greed Index - fetchFearGreed() (line 10767) 19
4.5 Halving Cycle Position - computeCyclePosition() (line 10952) 19

4.6 Astro Cycles - runAstroAnalysis() (line 11805) 20
4.7 ETF Flows - loadEtfFlows() (line 14426) 20
4.8 News Feed (line 15949-16105) 20
5. Best Trades Tab - AI Probability Engine 21
5.1 Scan Pipeline - runProbEngine() (line 14000) 21
5.2 Live Signal Computation - computeLiveSignals() (line 12983) 21
5.2.1 Indicator Signal Details 21
5.3 Confluence Scoring - scoreConfluence() (line 13551) 21
5.3.1 Timeframe-Adaptive Weights 22
5.3.2 Indicator Family Correlation Dampening 22
5.3.3 Trend vs Mean-Reversion Conflict 22
5.3.4 Base Probability (Sigmoid) 22
5.3.5 Contextual Adjustments (Priority Waterfall, line 13677) 22
5.4 Probability Calibration (line 13342-13469) 23
5.5 Risk/Reward Estimation - estimateRR() (line 13819) 23
5.6 Cross-Timeframe Analysis - fetchCrossTFBias() (line 13534) 23
5.7 Auto-Trade Execution (line 15515-15935) 23
6. Alerts Tab 24
6.1 Alert Types 24
6.2 Notification Channels 24
6.2.1 Browser Notifications (line 8320) 24
6.2.2 Discord Webhook (line 8287-8319) 24
6.2.3 Telegram Bot (line 8274-8454) 24
6.3 Alert Engine (line 17171-17456) 24
6.4 Alert Persistence 24
7. Paper Trade Tab 26
7.1 Strategy Management 26
7.1.1 showPaperTradePopup() (line 18069) 26
7.1.2 confirmPaperTrade() (line 18108) 26
7.1.3 startPaperTrading() (line 18138) 26
7.2 Strategy Polling - pollOneStrategy() (line 18333) 26
7.3 Risk Controls 26
7.4 Go-Live Graduation - goLiveFromPaper() (line 19035) 26
8. Live Trading Tab 28
8.1 Credential Management 28
8.1.1 liveStoreCredentials() (line 19916) 28

8.1.2 liveUnlockCredentials() (line 19951) 28
8.1.3 Wallet System (line 19094-19259) 28
8.2 API Integration 28
8.2.1 liveHeaders() (line 19744) 28
8.2.2 Key Functions 28
8.3 Safety Controls 28
8.3.1 Kill Switch - liveKillSwitch() (line 20205) 29
8.3.2 Safety Settings - liveSaveSafety() (line 20187) 29
8.4 Order Execution 29
8.4.1 BloFin Order Modal (line 19330) 29
8.4.2 Auto-Balance Refresh (line 19896) 29
9. Predictions Tab 30
9.1 Market Loading - loadPredictionMarkets() (line 20268) 30
9.2 Bot Strategies 30
9.2.1 BTC Sniper 30
9.2.2 Yes/No Arbitrage 30
9.3 Configuration 30
9.4 Performance Tracking 30
9.5 Bot Control 30
10. Watchlist, Portfolio & P100 System 31
10.1 Watchlist (line 12037-12460) 31
10.2 Bought List (line 12485-12935) 31
10.3 Perfect 100 System (line 16593-16910) 31
10.4 Scanner Accuracy Tracking (line 12230) 32
11. Recommendations & Tax Reporting 33
11.1 Trade Recommendations (line 14700-15375) 33
11.2 Portfolio Settings (line 14796-14845) 33
11.3 BloFin Trade History (line 14845-14977) 33
11.4 Tax Reporting (line 14977-15216) 33
12. localStorage Keys Reference 34

1. Architecture Overview

The Ultimate Crypto Backtester Pro is a monolithic single-page application (SPA) built entirely in a single index.html file (~20,500 lines). It combines HTML structure, CSS styling, and JavaScript logic into one deployable unit served by a minimal Node.js static file server (serve.js on port 3000).

1.1 Technology Stack

- Frontend: Vanilla JavaScript (no framework), HTML5, CSS3 with CSS custom properties
- Charting: Chart.js 4.4.0 (backtester charts), Lightweight Charts 4.1.3 (price charts)
- Fonts: Google Fonts - Syne (headings) + Space Mono (data)
- Backend: Node.js/Express hosted on Railway (authentication, paper trading, live trading)
- Exchange API: BloFin (futures trading, 530+ pairs)
- Data Sources: Binance (spot/futures candles), DexScreener (Solana meme coins), RugCheck (contract safety)
- Storage: localStorage for all client-side persistence (settings, watchlists, learned data)
- Notifications: Browser Notifications API, Discord webhooks, Telegram Bot API

1.2 Tab Structure

The dashboard is organized into 8 main tabs, each activated via switchPageTab(tabName):

Function	Line	Description
`Backtester`	backtester	Historical strategy testing with 14+ indicators, optimizer, walk-forward analysis, Pine Script export
`Degen Scanner`	degen	Solana meme coin scanner with 3-layer scoring, RugCheck integration, adaptive learning, auto-trade
`Market Intel`	intel	Market cycle analysis, Fear & Greed index, ETF flows, coin screener, astro cycles, news feed
`Best Trades`	trades	Real-time AI probability engine with 7-indicator confluence scoring, cross-TF analysis, auto-trade
`Alerts`	alerts	Price/indicator/scan-based alerts with browser notifications, Discord webhooks, Telegram
`Paper Trade`	paper	Forward-test strategies with simulated capital, risk controls, kill switch, multi-strategy support
`Live Trading`	live	BloFin API integration for real futures trading, position management, encrypted credential storage
`Predictions`	predictions	Polymarket prediction market bot with edge detection, BTC sniper, Yes/No arbitrage

1.3 Global Systems

1.3.1 Authentication (line 674-927)

Login/registration via backend API. Functions: switchAuthView(), tryLogin(), submitAuth(), authLogout(), showLoggedIn(). Subscription tiers (Free/Pro/Ultra) gate access to features. Backend fetch wrapper: backendFetch(method, path, body) at line 8939.

1.3.2 Live Price Ticker (line 4459-4474)

Scrolling ticker bar showing real-time prices for major pairs. fetchTicker() polls Binance API every 60 seconds. Displays symbol, price, and 24h change percentage.

1.3.3 Market Cycle Dashboard (line 3194-3872)

Persistent header section showing BTC cycle position. calcCycleDashboard(d) computes a composite score (0-100) from 7 sub-indicators, each weighted:

Function	Line	Description
`200-Day MA`	3209	Weight 20. Maps price-vs-MA percentage to score. Below MA = accumulation (low score). 300%+ above = euphoria (high score).
`RSI (14-period)`	3239	Weight 10. Direct mapping: RSI value = score. <30 oversold (buy zone), >70 overbought (sell zone).
`Pi Cycle Top`	3254	Weight 15. 111-day MA vs 2x350-day MA. Cross = cycle top signal. Gap percentage maps to risk score.
`2Y MA Multiplier`	3290	Weight 5. Price / 2-year MA. <1x = extreme bottom. >5x = historical top zone.
`ATH Drawdown`	3314	Weight 15. Percentage below all-time high. -75%+ = capitulation buy zone. Near ATH = top risk.
`Volume Trend`	3352	Weight 5. Up-volume vs down-volume ratio over 40 bars. >1.4 = accumulation. <0.8 = distribution.
`Trend Direction`	3381	Weight 15. 30/60/90-day momentum blend with MA position and higher-high/lower-low structure detection.

The composite score determines the cycle phase label: Deep Bear (0-20), Early Bull (20-40), Mid Bull (40-60), Late Bull (60-80), Euphoria (80-100). Also integrates halving cycle day tracking (halvingCycleDay = days since last halving, currently Day 694 post April 2024 halving).

2. Backtester Tab

The Backtester is the core strategy testing engine. It lets users select any Binance-listed pair, choose a timeframe, configure technical indicators, and simulate historical trades with realistic position sizing, leverage, stop-loss/take-profit, and performance metrics.

2.1 Data Loading

2.1.1 fetchKlines() (line 4662)

Fetches historical candlestick data from Binance REST API. Supports pagination for large date ranges (1000 candles per request). Each candle is normalized to { t: timestamp, o: open, h: high, l: low, c: close, v: volume }. Auto-loads BTC/USDT 1D data on first visit via autoLoadBTCDaily() (line 9091).

2.1.2 fetchBlofinCandles() (line 4514)

Alternative data source from BloFin exchange. Converts BloFin timeframe notation (e.g., '1H' vs '1h') and instrument IDs. Used when BloFin is the selected market source.

2.1.3 fetchStockKlines() (line 4550)

Fetches stock/ETF data when crypto markets are not selected. Uses Yahoo Finance proxy via backend.

2.1.4 resampleCandles() (line 4604)

Resamples candle data to higher timeframes by aggregating N candles. Preserves OHLCV structure with correct high/low extremes and summed volume.

2.2 Technical Indicators

All indicators are implemented as pure functions that take price arrays and return signal arrays. Each returns { name, buy[], sell[] } where buy/sell are boolean arrays aligned to the input data.

2.2.1 Core Math Functions (line 4059-4068)

Function	Line	Description
`sma(a, l)`	4059	Simple Moving Average. Sliding window sum over length l.
`ema(a, l)`	4060	Exponential Moving Average. Smoothing factor k = 2/(l+1). Recursive: EMA[i] = price*k + EMA[i-1]*(1-k).
`rsi(c, l)`	4061	Relative Strength Index. Wilder smoothing. avg_gain and avg_loss tracked with (prev*(l-1)+current)/l. RSI = 100 - 100/(1+RS).
`stdev(a, l)`	4062	Standard deviation over sliding window of length l. Population variance (divide by l, not l-1).

`atr(d, l)`	4063	Average True Range. TR = max(H-L, |H-prevC|, |L-prevC|). Then EMA of TR.
`donchianMid(d, p)`	4065	Donchian Channel midline: (highest high + lowest low) / 2 over period p.

2.2.2 Signal Generators (line 4079-4211)

Function	Line	Description
`buildRSISignals(d)`	4079	RSI crossover signals. Buy when RSI <= buy threshold (default 30). Sell when RSI >= sell threshold (default 70). Period from UI input.
`buildEMASignals(d)`	4080	EMA crossover. Buy on fast EMA crossing above slow EMA. Sell on fast crossing below. Default: 12/26.
`buildMACDSignals(d)`	4081	MACD line/signal crossover. MACD = EMA(fast) - EMA(slow). Signal = EMA(MACD, signal period). Buy/sell on crossovers.
`buildBBSignals(d)`	4082	Bollinger Bands. Buy when price touches lower band (SMA20 - 2*stdev). Sell at upper band (SMA20 + 2*stdev).
`buildTDSignals(d)`	4083	TD Sequential. Counts consecutive closes below/above close[i-4]. Buy signal on count 9 (bearish exhaustion). Sell on count 9 (bullish exhaustion).
`buildStochRSISignals(d)`	4085	Stochastic RSI. Applies stochastic formula to RSI values over 14 periods. K = (RSI-min)/(max-min)*100. Buy < 20, Sell > 80.
`buildIchimokuSignals(d)`	4088	Ichimoku Cloud. Tenkan-sen (9), Kijun-sen (26), Senkou A/B (52). Buy: price above cloud + TK cross. Sell: below cloud + TK cross.
`buildSupertrendSignals(d)`	4119	Supertrend indicator. Uses ATR-based bands. Buy when price crosses above lower band. Sell when crosses below upper band.
`buildVWAPSignals(d)`	4146	Volume-Weighted Average Price. Buy when price crosses above VWAP. Sell when crosses below.
`buildOBVSignals(d)`	4164	On-Balance Volume. Cumulative volume direction. Buy/sell on OBV vs its own SMA crossovers.
`buildADXSignals(d)`	4185	Average Directional Index. Measures trend strength. Buy when +DI > -DI and ADX > 25. Sell when -DI > +DI and ADX > 25.

2.3 Signal Combination

2.3.1 combineSignals() (line 5119)

Combines multiple indicator signals using configurable logic: AND (all must agree), OR (any signal triggers), MAJORITY (>50% agree), WEIGHTED (uses indicator-specific weights).

2.3.2 combineCrossTFSignals() (line 5074)

Cross-timeframe signal combination. Loads data for secondary timeframes, builds indicator signals on each, then combines using MTF Logic: Confirm Primary (AND), Independent (OR), or Majority.

2.3.3 buildTimelineIndex() (line 5055)

Aligns candles from different timeframes by timestamp. Maps each base-timeframe bar to the corresponding higher-timeframe bar for cross-TF analysis.

2.4 Backtest Engine

2.4.1 backtest() (line 4211)

Core simulation engine. Iterates through each bar, managing long and short positions with:

- Position sizing: Fixed percentage, Kelly Criterion, or Signal Quality adaptive sizing
- Stop-loss: Leverage-adjusted percentage. Triggers on intra-bar low (long) or high (short)
- Take-profit: Leverage-adjusted percentage. Triggers on intra-bar high (long) or low (short)
- Direction control: Long-only, Short-only, or Both
- Compounding: Profits reinvested into next trade (position sized from current cash)
- Mark-to-market equity tracking: Unrealized P&L included in equity curve

Returns: { final, ret (%), trades[], wr (%), equity[], maxDD, sharpe, slCount, tpCount, avgWin, avgLoss, kelly, longCount, shortCount, longWR, shortWR }

2.4.2 Position Sizing Modes

Fixed Percentage: Uses tradePct (default 100%) of current cash per trade, capped by maxTradePct.

Kelly Criterion: calcKelly(wr, avgWin, avgLoss) at line 4863. Kelly% = WR - (1-WR)/payoff. Capped at half-Kelly for safety. Applied as percentage of cash.

Signal Quality: Scales position from 50-100% of tradePct based on quality score (0-10). Higher quality signals get larger positions.

2.4.3 walkForwardTest() (line 4386)

Walk-forward analysis divides data into N windows. For each window, optimizes parameters on in-sample data (70%), then tests on out-of-sample data (30%). Validates strategy robustness by checking if optimized parameters work on unseen data. Renders comparison chart via renderWalkForward().

2.5 Optimizers

2.5.1 Leverage Optimizer (line 7694)

runLeverageOptimizer() tests leverage values from 1x to 125x (or user-specified range). For each leverage level, runs full backtest and records return, max drawdown, Sharpe ratio, and win rate. Finds optimal leverage that maximizes risk-adjusted returns. Results rendered as interactive chart.

2.5.2 SL/TP Optimizer (line 7759)

runSlTpOptimizer() grid-searches stop-loss (0.5%-20%) and take-profit (0.5%-50%) combinations. Scores each combination using configurable metric (Sharpe, Return, or Win Rate). Renders heatmap of results via renderSlTpOptimizer(). One-click apply with applySlTpOpt().

2.5.3 DCA Comparison (line 8030)

runDCAComparison() compares three strategies over the same period: Pure DCA (equal buys), Intelligent DCA (weighted by market quality signals), and Lump Sum. Renders comparison chart showing equity curves.

2.6 Market Quality & Regime Detection

2.6.1 buildSignalQuality() (line 4877)

Grades each bar A/B/C/No-Trade based on: ATR expansion (volatility), volume participation, EMA spread (trend clarity), Bollinger Band squeeze, and indicator alignment. Returns quality array (0-10) aligned to data.

2.6.2 detectRegime() (line 4905)

Classifies market regime per bar: trending-up, trending-down, ranging, or volatile. Uses combination of ADX, EMA slope, and volatility metrics. Used to adapt backtest behavior per regime.

2.7 Charts & Visualization

2.7.1 drawPriceChart() (line 6159)

Renders interactive price chart using Lightweight Charts library. Displays candlesticks with colored buy/sell markers. Overlays EMA lines, Bollinger Bands, and trade entry/exit points.

2.7.2 drawIndicatorSubPanels() (line 6336)

Renders sub-charts below main chart for each active indicator. Custom canvas rendering for RSI, MACD, EMA, BB, OBV, ADX with threshold lines and signal markers.

2.7.3 drawPNLChart() (line 7170)

Portfolio equity curve chart using Chart.js. Shows cumulative P&L over time with colored regions for drawdown periods.

2.7.4 drawTradesTable() (line 7202)

Sortable HTML table of all trades with columns: Type, Entry/Exit date and price, P&L, Leverage %, Win/Loss, Exit Reason. Sortable by any column via setSortKey().

2.8 Pine Script Export (line 7396)

Generates TradingView Pine Script v5 code from the current strategy configuration. Maps each indicator to its Pine Script equivalent with exact parameters. Includes strategy directives, risk management, and alert conditions. Output is copyable to TradingView directly.

2.9 AI Strategy Analyst (line 5819)

generateStrategyExplainer() sends backtest results to backend AI (Claude API) for plain-English analysis. Includes: what worked, what didn't, suggested improvements, risk assessment. Falls back to buildLocalExplainer() (line 5942) when offline, which generates analysis using rule-based logic.

2.10 Funding Rate Monitor (line 7004-7149)

loadLiveFunding() fetches real-time funding rates from BloFin for BTC, ETH, SOL, BNB and 30+ altcoins. Displays annualized rates with color coding: Green (neutral <0.05%), Yellow (elevated 0.05-0.1%), Red (extreme >0.1%). Negative = shorts paying longs. Historical funding overlay available on price chart.

3. Degen Scanner Tab

The Degen Scanner is a Solana meme coin discovery and evaluation engine. It scans DexScreener for newly listed tokens, scores them using a multi-layer system, checks contract safety via RugCheck, and optionally auto-trades qualifying tokens through Jupiter Aggregator.

3.1 Data Sources & Scanning

3.1.1 runClientDegenScan() (line 9655)

Main scan function. Fetches token data from DexScreener API for Solana pairs. Filters by minimum liquidity ($5K) and age. Scores each token with clientScoreToken(), enriches top candidates with RugCheck data, applies safety gates, and sorts by degenScore. Auto-refreshes every 5 minutes.

3.1.2 dexFetch() (line 9194)

Wrapper for DexScreener API calls with 8-second timeout and error handling.

3.1.3 rugFetch() (line 9205)

Fetches RugCheck.xyz report for a token address. Returns safety assessment including mint authority, freeze authority, LP status, top holder concentration.

3.1.4 parseRugReport() (line 9220)

Parses RugCheck API response into structured safety data: rugChecked (bool), rugScore (0-1000), mintDisabled, freezeDisabled, lpBurned, lpLocked, topHolderPct, isHoneypot, devSelling, taxBuy/taxSell percentages.

3.2 Token Scoring - clientScoreToken() (line 9338)

The primary scoring function evaluates each token on multiple dimensions, producing a degenScore (0-100+). Each dimension contributes positive or negative points:

3.2.1 Liquidity (line 9369)

$50K-500K: +18 points (Strong liquidity - optimal range)

$10K+: +12 points (Good)

$5K-10K: +6 points (Thin - warning)

<$5K: -20 points (Too thin - dangerous)

3.2.2 Volume/Market Cap Ratio (line 9375)

Age-adjusted thresholds. For tokens <48h old, higher ratios expected:

Legendary (>10x or >4x if older): +25 points

Strong (>6x or >2x): +20 points
Good (>3x or >1x): +13 points
Dead (<1x or <0.5x): -12 points

3.2.3 Volume Acceleration (line 9387)

Compares 1h volume run-rate vs 6h average. Detects whether momentum is building or fading.

Surging (>2.5x with prior activity): +15 points
Dead-to-pump spike (>2.5x without prior activity): -10 points (rug pattern flag)
Building (>1.5x with activity): +8 points
Volume dying (<0.3x): -12 points

3.2.4 Wash Trade Detection (line 9399)

Analyzes average volume per transaction as proxy for bot activity:

Avg <$200/tx: +12 points (Retail crowd - organic)
Avg >$15K/tx: -18 points (Bot volume - wash trading)
Avg >$5K/tx: -8 points (Possible wash)
Also checks total transaction count: >2000 txns = +8 pts. <30 txns with >$50K vol = -12 pts (suspicious).

3.2.5 Buy Pressure (line 9418)

1-hour buy/sell ratio analysis with manipulation detection:

>90% buys: -12 points (Likely manipulation - 82.8% of sub-$200K tokens with 90%+ buys are manipulated)
75-89% buys: +15 points (Strong buying)
65-74% buys: +20 points (Healthy - organic sweet spot)
55-64% buys: +10 points (Organic)
<40% buys: -15 points (Sell pressure)
90%+ buys + large avg tx: Additional -8 points (coordinated pump)

3.2.6 Age Sweet Spot (line 9432)

<30 minutes: -25 points (Sniper risk)
4-12 hours: +18 points (DNA-validated optimal entry window)
1-4 hours: +10 points (Early)
12-48 hours: +6 points (Still early)
>7 days: -10 points (Aged)

3.2.7 Market Cap Band (line 9441)

<$100K: +18 points (Micro-cap - maximum upside)

<$500K: +14 points

<$2M: +10 points

>$100M: -12 points (Limited upside)

3.2.8 Additional Scoring Factors

Price Momentum (line 9448): 1h change (+14 for >30%, -12 for <-25%), 24h change, momentum alignment bonus

Social Presence (line 9460): Twitter + Telegram + Website = +12. No socials = -10

Social Velocity (line 9470): New token (<24h) + socials + high tx = +8 (viral interest proxy)

Narrative Matching (line 9480): scoreNarrative() checks symbol/name against trending themes (AI, meme, dog, cat, etc.). Capped at +12

3.3 Probability Calibration - calcMemeProbability() (line 10327)

Converts degenScore into calibrated probabilities for 2x, 5x, and 10x returns using sigmoid functions:

Formula: P = 1 / (1 + exp(-0.10 * (score - midpoint)))

Midpoints calibrated to real meme coin base rates:

- 2x probability: midpoint 50, ceiling 82% (base rate ~15%)
- 5x probability: midpoint 70, ceiling 50% (base rate ~5%)
- 10x probability: midpoint 78, ceiling 30% (base rate ~2%)

Adjustments applied on top of base sigmoid:

- Historical success rates from tracked P100/Watchlist tokens (up to 50% blend weight)
- Feature-specific boosts via adaptive weights from EWMA learning engine
- Vol/Mcap ratio >8x: +6%, Buy pressure >73%: +5%, Sweet spot age: +4%
- Penalties: Low buy pressure (<50%): -8%, Dead volume: -6%, Aged (>7d): -4%

3.4 RugCheck Integration

After initial scoring, top 50 tokens are enriched with RugCheck data via rugFetch(). The parseRugReport() function extracts:

- Mint authority status (disabled = safe)
- Freeze authority status (disabled = safe)
- LP burn/lock status
- Top holder concentration percentage

- Honeypot detection
- Developer selling activity
- Buy/sell tax percentages

Hard-blocks (_hardBlocked = true) are set for: honeypot, blacklisted, freeze authority enabled, >90% single holder. Auto-trade blocks (_autoTradeBlocked) for: active dev selling, >10% buy/sell tax.

3.5 Token Tiers & Display

3.5.1 buildDegenTiers() (line 9606)

Categorizes scored tokens into return-potential tiers:

- 1000x Candidates: Micro-cap (<$200K), early, proven demand - modeled on BONK/FARTCOIN at graduation
- 100x Candidates: Sub-$1M mcap, strong metrics
- 10x Candidates: Sub-$10M mcap, solid fundamentals
- 2-5x Candidates: Established tokens with upside

3.6 Smart Money Detection - checkSmartMoney() (line 16496)

Tracks wallet addresses that made profitable trades on previous winning tokens. Requirements:

- Minimum 2 winning trades at 5x+ to qualify as smart money wallet
- Confidence decays with 2-week half-life: decay = 0.5^(ageDays/14)
- Wallets stale >4 weeks (confidence <25%) are skipped
- Returns average confidence of matching smart money wallets

3.7 Auto-Trade System - checkDegenAutoTrade() (line 16200)

Automated meme coin trading with comprehensive safety gates:

3.7.1 Safety Gates (in order of evaluation)

- Daily drawdown limit: 5% of starting balance - pauses all meme auto-trades for the day
- Rolling 7-day drawdown: 10% - pauses auto-trades for the week
- Hard-blocked tokens skipped (honeypot, blacklist, freeze)
- Auto-trade-blocked tokens skipped (dev selling, high tax)
- Minimum degenScore threshold (configurable)
- Minimum 2x probability threshold (configurable)
- Maximum market cap filter
- Minimum buy pressure filter
- Minimum vol/mcap ratio filter

- RugCheck must pass: rugChecked = true
- LP must be burned or locked
- Price impact check: trade size / liquidity must be < 2%
- Top holder concentration must be < 40%

3.7.2 Position Sizing & Exit Strategy

Position size: 1.5% of available balance per meme trade (capped by configured max)
Exit strategy (laddered): 40% at 2x, 30% at 5x, 20% at 10x, 10% moonbag
Stop-loss: -30% (wider than typical due to meme coin volatility)
Execution: Opens Jupiter Aggregator (jup.ag) swap page for the token

3.8 Adaptive Learning Engine (line 16310-16475)

EWMA-based weight adjustment system that learns from tracked token outcomes:

Storage: localStorage key 'degenAdaptive_v2'
Smoothing factor (alpha): 0.2
Minimum samples before adjusting: 10 per bucket
Weight range: 0.5x to 2.0x multiplier

Feature buckets tracked:

- volMcap: low (<2x), medium (2-6x), high (>6x)
- buyRatio: weak (<60%), good (60-73%), strong (>73%)
- age: early (<12h), mid (12-48h), old (>48h)
- mcap: micro (<$100K), small ($100K-1M), large (>$1M)

For each bucket, tracks success rate (2x/5x/10x hits) with exponential smoothing. When a bucket's success rate exceeds the overall average, its scoring weight increases (up to 2x). Below average = weight decreases (down to 0.5x). Applied via getAdaptiveWeights() in calcMemeProbability().

3.9 Sub-tabs

- By Return Tier: Tokens grouped by 1000x/100x/10x/2-5x potential
- Live Feed: Real-time token discovery stream (startLiveFeed/fetchLiveFeed)
- DNA Benchmarks: Historical performance analysis of scored tokens (showDNAModal)
- Watchlist: User-tracked tokens with milestone detection (renderWatchlist)
- Bought: Logged purchases with sell alerts (renderBoughtList, checkSellAlerts)
- Perfect 100: Auto-captured high-scoring tokens (autoCaptureP100, renderPerfect100)

- 100x Club: Tokens that achieved 100x returns (checkHall100xPromotion, renderHall100x)

4. Market Intel Tab

The Market Intel tab provides macro-level market analysis across multiple data dimensions. It aggregates data from external APIs to provide a comprehensive market overview.

4.1 Market Overview - renderOverview() (line 11073)

Displays key market indicators in card format:

- Total Market Cap (from CoinGecko global API)
- BTC Dominance percentage
- ETH Dominance percentage
- Active Cryptos count
- Total 24h Volume
- Halving Cycle Day (days post-halving, historical comparison)
- BTC and ETH current prices

4.2 Coin Screener - renderScreener() (line 11207)

Sortable table of top coins with computed scores. scoreCoIn() (line 10807) evaluates each coin on: price momentum (1h/24h/7d change), volume/mcap ratio, market cap rank, supply metrics. Includes sortScreener() for column sorting and applyScreenerFilter() for filtering by criteria.

4.3 AI Top Picks - generateAIPicks() (line 11280)

Sends market data to backend AI for analysis. Returns ranked list of coins with buy/sell/hold recommendations, confidence scores, and reasoning. Falls back to client-side scoring when offline.

4.4 Fear & Greed Index - fetchFearGreed() (line 10767)

Fetches Alternative.me Fear & Greed Index. Displays current value (0-100), yesterday's value, and 7-day trend. Color-coded gauge: Extreme Fear (0-25), Fear (25-45), Neutral (45-55), Greed (55-75), Extreme Greed (75-100).

4.5 Halving Cycle Position - computeCyclePosition() (line 10952)

Maps current day post-halving to historical cycle phases: Consolidation (0-180d), Recovery (180-365d), Bull (365-480d), Peak Zone (480-550d), Distribution (550-800d). Compares with previous cycles (2012, 2016, 2020) for pattern recognition.

4.6 Astro Cycles - runAstroAnalysis() (line 11805)

Planetary cycle analysis overlaid on price data:

- Lunar phases: calcLunarPhases() (line 11535) - New Moon/Full Moon correlation with price
- Planetary conjunctions and aspects computed via buildAllAstroEvents() (line 11761)
- Historical impact analysis: analyseAstroImpact() (line 11664) - computes average returns after each event type
- Julian Date conversion: dateToJD/jdToDate (line 11524)

4.7 ETF Flows - loadEtfFlows() (line 14426)

Tracks Bitcoin and Ethereum spot ETF flow data:

- switchEtfAsset() toggles between BTC and ETH ETF views
- renderEtfFlows() displays daily inflow/outflow chart
- renderEtfChart() renders cumulative flow chart
- renderEtfIssuerBreakdown() shows per-issuer breakdown (Blackrock, Fidelity, etc.)
- updateEtfSummary() computes running totals and trend

4.8 News Feed (line 15949-16105)

Real-time crypto news aggregation:

- loadNews() fetches from backend news API
- connectNewsStream() opens SSE connection for live updates with exponential backoff
- renderNewsFeed() displays articles with source, timestamp, sentiment badge
- updateNewsSentiment() analyzes article sentiment distribution
- filterNews(category) filters by: All, Bitcoin, Ethereum, DeFi, NFT, Regulation
- extractCoinFromTitle() identifies mentioned coins for linking

5. Best Trades Tab - AI Probability Engine

The Best Trades tab is a real-time trade signal generator that scans 20 crypto assets, computes multi-indicator confluence scores, calibrates probabilities, and optionally auto-executes trades via BloFin.

5.1 Scan Pipeline - runProbEngine() (line 14000)

End-to-end signal generation pipeline:

- 1. Fetches 200 candles for selected timeframe for each of 20 tracked assets
- 2. Computes all 7 indicators per asset: RSI, EMA, MACD, BB, StochRSI, Ichimoku, Volume
- 3. Detects current market regime per asset
- 4. Fetches cross-timeframe bias data
- 5. Computes confluence score for both long and short directions
- 6. Calibrates raw probability using historical accuracy data
- 7. Estimates risk/reward ratio with dynamic SL/TP
- 8. Applies correlation discount for correlated pairs
- 9. Renders trade cards sorted by probability

5.2 Live Signal Computation - computeLiveSignals() (line 12983)

Computes all 7 indicator signals for a given asset and timeframe. Each indicator returns bull/bear/neutral with detailed labels and cross-detection:

5.2.1 Indicator Signal Details

RSI (14): Bull if RSI < 35 (oversold). Bear if RSI > 65. Cross-signal on threshold crossing within last 2 bars.

EMA (12/26): Bull if fast EMA > slow EMA. Bear if fast < slow. Cross-signal on actual crossover.

MACD (12/26/9): Bull if MACD line > signal line. Bear if below. Cross-signal on line crossover.

Bollinger Bands (20,2): Bull if price at lower band (mean-reversion buy). Bear at upper band. Squeeze detection when bandwidth < 20% of average.

StochRSI (14): Bull if K < 25 (oversold). Bear if K > 75. Cross-signal on threshold crossing.

Ichimoku (9/26/52): Bull if price above cloud + TK cross up. Bear if below cloud + TK cross down. Cloud thickness indicates support/resistance strength.

Volume: Bull if volume > 1.5x 20-period average on up-bar. Bear if high volume on down-bar. Drying flag if volume < 0.5x average.

5.3 Confluence Scoring - scoreConfluence() (line 13551)

Combines indicator signals into a single probability score with sophisticated weighting:

5.3.1 Timeframe-Adaptive Weights

Indicator weights change based on timeframe to reflect signal reliability:

Function	Line	Description
`Short TF (1m-15m)`	-	RSI: 22, StochRSI: 22, BB: 18, Volume: 15, MACD: 10, EMA: 8, Ichimoku: 5
`Medium TF (30m-1h)`	-	RSI: 18, StochRSI: 18, EMA: 14, MACD: 14, BB: 14, Volume: 12, Ichimoku: 10
`Default (2h-4h)`	-	EMA: 20, Ichimoku: 16, MACD: 16, RSI: 12, Volume: 14, BB: 12, StochRSI: 10
`Long TF (6h-1w)`	-	EMA: 25, Ichimoku: 22, MACD: 18, RSI: 10, Volume: 10, StochRSI: 8, BB: 7

5.3.2 Indicator Family Correlation Dampening

Indicators are grouped into families to prevent triple-counting correlated signals:

- Mean-Reversion family: RSI, StochRSI, BB (all derive from price mean-reversion)
- Trend-Following family: EMA, MACD, Ichimoku (all measure trend direction)
- Flow family: Volume (independent)

Diminishing returns within each family: 1st signal = 100%, 2nd = 60%, 3rd = 35%

5.3.3 Trend vs Mean-Reversion Conflict

When 2+ trend indicators disagree with mean-reversion indicators (e.g., RSI oversold but EMA/MACD bearish), mean-reversion signals are halved. This prevents false bottom-picking in strong downtrends.

5.3.4 Base Probability (Sigmoid)

Raw confluence ratio (0-1) is mapped through sigmoid: prob = 28 + 50 / (1 + exp(-7 * (confluence - 0.5)))

This maps 0% confluence to ~28% probability, 50% confluence to ~53%, and 100% confluence to ~78%.

5.3.5 Contextual Adjustments (Priority Waterfall, line 13677)

Applied in order of predictive value with per-factor sub-caps and global cap of +/-20 points:

- Cross-timeframe alignment: +/- up to 8 pts based on HTF trend agreement
- Market regime: trending regime boosts trend signals, ranging boosts mean-reversion
- Funding rate: extreme negative funding = long bias boost, extreme positive = short bias

- Market quality grade: A-grade = +5, C-grade = -5

5.4 Probability Calibration (line 13342-13469)

Tracks prediction accuracy over time to adjust future probabilities:

- recordCalibrationPrediction(): Logs each prediction with asset, timeframe, probability, direction, entry/stop/target prices
- resolveCalibrationPredictions(): Checks if predictions hit target or stop. Records outcome.
- getCalibrationStats(): Returns accuracy per probability bucket (50-60%, 60-70%, etc.)
- getCalibratedProb(): Adjusts raw probability based on historical accuracy for that probability range and regime

5.5 Risk/Reward Estimation - estimateRR() (line 13819)

Computes dynamic stop-loss and take-profit levels:

- Stop-loss: Based on ATR * multiplier, adjusted by market quality and regime. Minimum 0.5%, maximum 15%.
- Take-profit: Based on probability * R:R ratio target. Higher probability = tighter TP (more realistic). Lower probability = wider TP.
- Uses swing points from findSwingPoints() (line 13804) for structural support/resistance levels.
- Kelly Criterion applied for position sizing recommendation.

5.6 Cross-Timeframe Analysis - fetchCrossTFBias() (line 13534)

Implements Elder's Triple Screen approach. For each asset, fetches one higher timeframe and computes trend bias via quickTrendBias() (line 13494). HTF alignment score modifies probability: strong agreement = +8 pts, strong disagreement = -8 pts.

5.7 Auto-Trade Execution (line 15515-15935)

When enabled, processAutoTrades() (line 15770) evaluates scan results against configured thresholds:

- Min probability threshold (default 70%)
- Position size (fixed dollar amount or % of balance)
- Max open positions limit
- Mode: Confirm First (shows popup) or Auto-Execute (immediate)
- autoExecDirect() (line 15844) places order via BloFin API with computed SL/TP
- Refresh positions after execution via refreshBtPositions() (line 15678)

6. Alerts Tab

The Alert Manager provides configurable alerts with multiple notification channels.

6.1 Alert Types

- Price Cross: Triggers when price crosses above/below a target level
- RSI Threshold: Triggers on RSI crossing overbought/oversold levels
- MACD Cross: Triggers on MACD line/signal line crossover
- EMA Cross: Triggers on fast/slow EMA crossover
- Volume Spike: Triggers when volume exceeds N times average
- Bollinger Squeeze: Triggers when BB bandwidth contracts below threshold
- Scan-Based: Triggered by Best Trades scan results above probability threshold
- Perfect 100: Triggered when Degen Scanner finds a 100-score token
- Milestone: Triggered when tracked tokens hit 2x/5x/10x milestones
- Strategy: Saved from backtester - triggers when live market matches backtest signal conditions

6.2 Notification Channels

6.2.1 Browser Notifications (line 8320)

enableBrowserNotifications() requests Notification API permission. Sends desktop notifications even when tab is in background.

6.2.2 Discord Webhook (line 8287-8319)

saveDiscordWebhook() stores webhook URL. sendDiscordWebhook() POSTs rich embed messages with alert details, probability, and action links.

6.2.3 Telegram Bot (line 8274-8454)

sendTelegram() sends messages via Telegram Bot API. saveTgCredentials() stores bot token and chat ID. Supports polling mode via startTelegramAlerts().

6.3 Alert Engine (line 17171-17456)

startAlertEngine() launches polling loop (every 60 seconds). runAlertPoll() iterates all active alerts, fetches required data (candles, indicators), evaluates conditions, and fires alerts. Cooldown tracking via amOnCooldown() prevents alert spam. Alert history stored in localStorage.

6.4 Alert Persistence

All alerts stored in localStorage (key: 'alertManager_v3'). Alert history in 'alertManager_hist_v1'. Telegram credentials in 'alertManager_tg_v1'. Functions: amLoad(), amSave(), amHistLoad(), amHistPush().

7. Paper Trade Tab

Paper Trading allows forward-testing strategies with simulated capital before risking real money. Strategies sync to the Railway backend for persistent execution even when the browser is closed.

7.1 Strategy Management

7.1.1 showPaperTradePopup() (line 18069)

Opens configuration modal for new paper strategy. Imports indicator settings from the backtester tab.

7.1.2 confirmPaperTrade() (line 18108)

Validates settings and registers strategy in localStorage via ptRegSave(). Strategy definition includes: symbol, timeframe, indicators, parameters, initial capital, lookback window.

7.1.3 startPaperTrading() (line 18138)

Activates strategy: initializes state (ptStateDefault), starts polling loop, syncs to backend. Backend continues running strategies independently.

7.2 Strategy Polling - pollOneStrategy() (line 18333)

Core execution loop for each active strategy:

- Fetches latest candles via ptFetchKlines() (line 18294)
- Rebuilds indicator signals from full candle history
- Checks for new entry/exit signals on latest bar
- Manages positions with SL/TP (same logic as backtester)
- Updates strategy state: equity, trade count, P&L
- Syncs state to backend every 60 seconds

7.3 Risk Controls

Max daily trades: Configurable limit (default 999)

Stop after N losses: Pauses strategy after N consecutive losses (default 3)

Kill Switch: Emergency stop that halts all strategies immediately

Lookback window: 30, 60, or 90 days of data for signal computation

7.4 Go-Live Graduation - goLiveFromPaper() (line 19035)

When paper strategy proves profitable (checkGoLiveUnlock at line 19009 - requires positive return after minimum trades), user can promote to live trading. Transfers strategy configuration to Live Trading tab.

8. Live Trading Tab

The Live Trading tab connects to BloFin exchange for real futures trading with full position management, encrypted credential storage, and safety controls.

8.1 Credential Management

8.1.1 liveStoreCredentials() (line 19916)

Encrypts API key, secret, and passphrase using AES-GCM via Web Crypto API. Stores encrypted blob in localStorage. Requires user-set encryption password.

8.1.2 liveUnlockCredentials() (line 19951)

Decrypts stored credentials with user password. Credentials remain in memory only during session.

8.1.3 Wallet System (line 19094-19259)

Optional trading wallet generation/import. Uses deriveKey() (PBKDF2), encryptBytes(), decryptBytes() for secure key management.

8.2 API Integration

8.2.1 liveHeaders() (line 19744)

Generates authenticated headers for BloFin API: API-KEY, ACCESS-SIGN (HMAC-SHA256), ACCESS-TIMESTAMP, ACCESS-PASSPHRASE.

8.2.2 Key Functions

Function	Line	Description
`liveRefreshAll()`	19752	Refreshes all live trading data: status, balance, strategies, positions, history, safety, markets
`liveRefreshBalance()`	19826	Fetches account balance from BloFin. Displays available/frozen/total USDT.
`liveRefreshPositions()`	20093	Fetches open positions. Displays unrealized P&L, entry price, leverage, liquidation price.
`liveRefreshHistory()`	20132	Fetches closed trade history with P&L summary.
`liveRefreshMarkets()`	20225	Loads 530+ BloFin futures pairs with search/filter.
`liveRefreshStrategies()`	20021	Lists active auto-trading strategies promoted from paper trading.
`connectLiveStream()`	19988	Opens WebSocket for real-time position and order updates.

8.3 Safety Controls

8.3.1 Kill Switch - liveKillSwitch() (line 20205)

Emergency function: closes ALL open positions, cancels ALL pending orders, deactivates ALL strategies. Sends market close orders via backend. Confirms with user before execution.

8.3.2 Safety Settings - liveSaveSafety() (line 20187)

Configurable safety parameters: max daily loss limit, max open positions, max leverage per trade. Enforced server-side by backend.

8.4 Order Execution

8.4.1 BloFin Order Modal (line 19330)

openBlofinModal(setup) displays order configuration: pair, direction, size, leverage, SL/TP. confirmBlofinExec() places order via backend API proxy. setBlofinLeverage() updates leverage setting.

8.4.2 Auto-Balance Refresh (line 19896)

startBalanceAutoRefresh() polls balance every 30 seconds when Live Trading tab is active.

9. Predictions Tab

The Predictions tab integrates with Polymarket prediction markets for automated trading based on edge detection.

9.1 Market Loading - loadPredictionMarkets() (line 20268)

Fetches active prediction markets from backend Polymarket proxy. Displays markets with current prices, volume, and end dates. Real-time updates via connectPredictionStream() SSE connection.

9.2 Bot Strategies

9.2.1 BTC Sniper

Monitors BTC-related prediction markets for pricing inefficiencies. Uses technical analysis from the Backtester engine to estimate true probability, then compares with market-implied probability.

9.2.2 Yes/No Arbitrage

Scans for markets where Yes + No prices sum to significantly less than $1 (guaranteed profit opportunity) or where one side is mispriced relative to fundamental analysis.

9.3 Configuration

Bet Size (USDC): Amount per trade (default $10)

Edge Threshold (%): Minimum edge required to trigger trade (default 10%)

Strategies: BTC Sniper and/or Yes/No Arb can be enabled independently

9.4 Performance Tracking

renderPredictionPerformance() (line 20400) shows win rate, total P&L, and trade log. Trade history persisted in localStorage.

9.5 Bot Control

startPredictionBot() (line 20435): Activates selected strategies, begins market monitoring

stopPredictionBot() (line 20455): Deactivates all strategies, stops monitoring

savePredictionConfig() (line 20475): Persists bet size, threshold, strategy selection to localStorage

10. Watchlist, Portfolio & P100 System

10.1 Watchlist (line 12037-12460)

Token tracking system integrated with Degen Scanner:

Function	Line	Description
`toggleWatchlist()`	12071	Add/remove token from watchlist. Stores address, symbol, metrics at time of addition.
`refreshWatchlistPrices()`	12139	Polls DexScreener for current prices of all watched tokens.
`checkMilestones()`	12195	Detects when watched tokens hit 2x, 5x, 10x, 50x, 100x milestones. Shows toast notification.
`renderWatchlist()`	12271	Displays watchlist with current price, P&L since addition, milestone badges.
`removeFromWatchlist()`	12460	Removes token and clears associated data.

10.2 Bought List (line 12485-12935)

Tracks actual purchases for P&L monitoring:

Function	Line	Description
`handleBuyClick()`	12495	Initiates buy logging flow. Records buy price, amount, timestamp.
`confirmBuy()`	12545	Confirms purchase entry with optional Jupiter link.
`checkSellAlerts()`	12607	Monitors bought tokens for sell conditions: -30% stop-loss, 2x/5x/10x targets, volume crash.
`markSold()`	12864	Marks token as sold, records exit price and P&L.
`renderBoughtList()`	12720	Displays portfolio with live P&L, gain percentage, sell alert status.

10.3 Perfect 100 System (line 16593-16910)

Automatic tracking of highest-scoring tokens:

Function	Line	Description
`autoCaptureP100()`	16593	Auto-saves tokens scoring 90+ to P100 list with capture metrics (score, mcap, vol/mcap, buy%, age).
`refreshP100Prices()`	16664	Updates current prices and checks for milestone achievements.
`checkHall100xPromotion()`	16705	Promotes tokens that achieve 100x returns to the Hall of Fame (100x Club).

`renderPerfect100()`	16766	Displays P100 tokens with capture vs current metrics, milestone badges, and performance tracking.

10.4 Scanner Accuracy Tracking (line 12230)

buildScannerAccuracy() computes historical accuracy of the scoring system. Tracks what percentage of high-scored tokens actually achieved 2x, 5x, 10x. Displayed in DNA Benchmarks modal.

11. Recommendations & Tax Reporting

11.1 Trade Recommendations (line 14700-15375)

Saves and tracks AI-generated trade recommendations:

Function	Line	Description
`saveRecommendation()`	14700	Stores recommendation with entry/exit/SL levels, probability, source (Best Trades or AI Picks).
`loadRecommendations()`	14726	Fetches saved recommendations from backend.
`renderRecSummary()`	14745	Displays summary stats: total recs, hit rate, average return.
`renderRecHistory()`	15348	Historical list of all recommendations with outcomes.
`logBestTrade()`	15375	Logs a Best Trades signal as a recommendation for tracking.

11.2 Portfolio Settings (line 14796-14845)

loadPortfolioSettings() and savePortfolioSettings() manage: initial capital, risk per trade percentage, max drawdown threshold. recomputePortfolio() recalculates based on updated settings.

11.3 BloFin Trade History (line 14845-14977)

loadBlofinTradeHistory() fetches closed trades from BloFin API. renderBlofinHistory() displays in sortable table. updateBlofinSummary() computes aggregate stats: total trades, win rate, net P&L, best/worst trade.

11.4 Tax Reporting (line 14977-15216)

Built-in tax report generation:

Function	Line	Description
`getTaxCache()`	14977	Loads cached trade data for tax computation.
`mergeTradesToCache()`	14986	Merges new trades into tax cache without duplicates.
`updateTaxUI()`	15019	Updates tax summary display with totals.
`getFilteredTaxTrades()`	15050	Filters trades by date range for tax year.
`renderTaxPreview()`	15062	Shows preview of taxable events.
`exportTaxCSV()`	15162	Exports tax data as CSV compatible with tax software.
`exportTaxPDF()`	15216	Generates PDF tax report with summary and trade details.

12. localStorage Keys Reference

The application uses localStorage extensively for client-side persistence. Key reference:

Function	Line	Description
`degenAdaptive_v2`	-	Adaptive learning EWMA weights and bucket data
`degenPendingOutcomes_v1`	-	Pending outcome tracking for learning engine
`degenDrawdown_v1`	-	Daily/rolling drawdown tracking for meme auto-trade
`degenAutoTrade_v1`	-	Meme auto-trade settings (enabled, thresholds, mode)
`degenAutoTradeLog_v1`	-	Auto-trade execution log (last 50 entries)
`smartWallets_v1`	-	Smart money wallet tracking data
`memeWatchlist_v2`	-	Degen Scanner watchlist tokens
`memeBought_v2`	-	Bought token records with purchase details
`memePerfect100_v2`	-	Auto-captured high-scoring tokens
`memeHall100x_v1`	-	100x Club hall of fame entries
`alertManager_v3`	-	All configured alerts
`alertManager_hist_v1`	-	Alert trigger history
`alertManager_tg_v1`	-	Telegram bot credentials
`discordWebhookUrl`	-	Discord webhook URL
`ptRegistry_v3`	-	Paper trading strategy definitions
`ptState_{id}`	-	Individual paper strategy state (equity, trades, position)
`blofinCreds_v1`	-	Encrypted BloFin API credentials
`walletData_v1`	-	Encrypted trading wallet data
`calibration_v2`	-	Probability calibration predictions and outcomes
`tfWinRateCache_v1`	-	Timeframe win rate cache for calibration
`btcDailyCache`	-	Cached BTC daily candle data
`predConfig_v1`	-	Prediction bot configuration
`autoTradeSettings_v1`	-	Best Trades auto-trade settings
`satsMode`	-	Sats stacking mode toggle
`taxCache_v1`	-	Cached trade data for tax reports

--- End of Methodology Document ---

Generated March 2026 | Ultimate Crypto Backtester Pro v2.0